# Recursive Algorithms

## Part 1: Introduction to Recursion

# Recursion

**Recursion** is when a function solves a problem by calling itself.

Each call works on a smaller or simpler version of the original problem.

The process continues until the function reaches a stopping condition.

Recursion is useful for problems that naturally contain smaller versions of themselves.

```
def countdown(number):
    print(number)

    if number > 1:
        countdown(number - 1)


countdown(3)
```

```
OUTPUT
3
2
1
```

# Recursive vs Iterative Algorithms

**Recursion** and **iteration** can often solve the same problems.

*An **iterative algorithm** repeats code using a loop.*

*A **recursive algorithm** repeats work by calling a function again.*

The best approach depends on the problem being solved

# Recursive vs Iterative Algorithms – Similarities

Both recursion and iteration **repeat a process**.

Both **need a condition** that eventually causes the repetition **to stop**.

Both can often produce **the same result**.

# Recursive vs Iterative Algorithms – Similarities

```
# Iterative Algorithm

def countdown(number):
    # Check condition before repeating
    while number > 0:
        # Print n
        print(number)
        # Assign n -= 1
        number -= 1

countdown(3)
```

```
OUTPUT
3
2
1
```

```
# Recursive Algorithm

def countdown(number):
    # Print n
    print(number)
    # Check condition before repeating
    if number > 1:
        # Repeat process with n - 1
        countdown(number - 1)

countdown(3)
```

```
OUTPUT
3
2
1
```

# Recursive vs Iterative Algorithms - Differences

***Note: For our count down algorithm, iteration is the better choice. However, it is a simple enough algorithm that it can be used for explanations.***

Later on in this course, we will focus more on algorithms that do work better with recursion such as:

## Solving a Maze

A maze can be solved by trying a path and continuing forward. If the path reaches another choice, the same process is repeated from that new position. If the path reaches a dead end, the program can return to an earlier choice and try a different path.

## Searching Through Folders

Folders can contain other folders. To search every folder inside a folder, the same search process can be repeated each time another folder is found. Recursion works well because each new folder is a smaller version of the original problem.

## Working with Tree Structures

Tree structures are made of items that can contain smaller branches of items. Examples include file systems, organization charts, and some game decision systems. Recursion works well because each branch can be processed using the same steps as the whole tree.

## Divide-and-Conquer Algorithms

Some problems can be split into smaller versions of the same problem. Each smaller problem is solved separately, and the results are combined. Algorithms such as merge sort use this idea by repeatedly dividing data into smaller pieces until the pieces become simple to solve.

# Recursive vs Iterative Algorithms - Differences

## Programmer Efficiency

Some problems are easier for a programmer to describe using recursion. This is especially true when the problem naturally contains smaller versions of itself. In these situations, recursive code can be shorter and easier to understand.

## Space Efficiency

Iteration normally uses less memory. Each recursive function call must be remembered until that call finishes. These unfinished calls are stored on the **call stack**.

## Time Efficiency

Iteration is often slightly faster because calling a function has extra work associated with it. Recursive solutions may create many function calls to solve a problem. However, the clearest algorithm is often more important than a small difference in speed.

# Difference - Programmer Efficiency

**Iteration:**

```
Start at a number.

Print the number.

Subtract 1 from the number.

Repeat while the number is greater than 1.
```

**In this example, iteration is probably easier to write and understand.**

**Recursion:**

```
Print the current number.

Check if we are still above 0.

If yes, call the same function to count down from one smaller number.
```

The function keeps solving a smaller version of the same problem until it reaches the base case.

# Difference - Space Efficiency

The **iterative** version only needs to keep track of the current value of `number`.

Each loop repeats using the same small amount of memory.

The recursive version creates a new function call every time it counts down by 1.

Python must remember each unfinished call on the **call stack**.

For `countdown(3)`, recursion needs to remember:

```
countdown(3)
countdown(2)
countdown(1)
countdown(0)
```

Because recursion stores all of these calls, it uses more memory than the iterative version.

# Differences – Time Efficiency

The iterative version repeats the loop once for each number.

Counting down from 3 requires about three repeated steps.

The recursive version also works through the numbers one at a time.

However, each step requires another function call.

Creating and returning from function calls takes some extra work.

For a simple countdown, iteration is usually a little faster.

Both versions still need roughly the same number of steps as the starting number gets larger.

# Recursive Algorithms Define Themselves in Terms of Themselves

A recursive algorithm uses the same process to solve a smaller version of its problem.

The function does not immediately solve everything at once.

Instead, it passes a smaller problem back to itself.

This continues until the problem becomes simple enough to stop.

**In this example, the important recursive idea is:**

`countdown(number - 1)`

The function **`countdown()`** uses another call to **`countdown()`** to continue solving the problem.

```
def countdown(number):
    print(number)
    countdown(number - 1)
```

# Recursive Algorithms Have a Base Case

A **base case** is the condition that stops recursion.

**When the base case is called, the function does not call itself again!**

Without a base case, the function would continue calling itself until Python eventually produces an error.

The base case represents the simplest version of the problem.

When `number` becomes `0`, the function returns instead of calling itself again.

```
def countdown(number):
    # Base Case: number == 0
    # Check the base case
    if number == 0:
        return
    # IF NOT THE BASE CASE:
    # Print number
    print(number)
    # Call function with n - 1
    countdown(number - 1)

countdown(3)
```

```
OUTPUT
3
2
1
```

# Recursive Algorithms Need to Move Toward the Base Case

Having a base case is not enough.

Each recursive call must also move the problem **closer to the base case**.

Otherwise, the base case may never be reached.

The important change is:

`number - 1`

Each call makes `number` smaller until eventually it reaches `0`.

```
def countdown(number):
    # BASE CASE: number == 0
    # Check the base case
    if number == 0:
        return
    # IF NOT THE BASE CASE:
    # Print number
    print(number)
    # Call function with n - 1
    countdown(number - 1)

countdown(3)
```

# The Recursive Step

The **recursive step** is the part of the function that calls itself again.

It usually changes the data before passing it into the next call.

This creates a smaller version of the same problem.

In this example:

`countdown(number - 1)`

is the recursive step.

```
def countdown(number):
    # BASE CASE: number == 0
    # Check the base case
    if number == 0:
        return
    # IF NOT THE BASE CASE:
    # Print number
    print(number)
    # RECURSIVE STEP
    # Call function with n - 1
    countdown(number - 1)

countdown(3)
```

# Tail End Recursion

**Tail end recursion** happens when the recursive call is the last operation performed by the function.

The current call has no more work to do after the recursive call finishes.

This can make the flow of the recursion easier to follow.

The recursive call is at the end:

`countdown(number - 1)`

Python does **not** automatically optimize tail recursion.

Each recursive call still uses space on the call stack while it is waiting to finish.

```
def countdown(number):
    # BASE CASE: number == 0
    if number == 0:
        return
    # PROCESS
    print(number)

    # RECURSIVE STEP
    countdown(number - 1)
    return
```

# Tail End Recursion - Using the Call Stack

Every time a function is called, Python needs to remember that function call.

These calls are stored in the **call stack**.

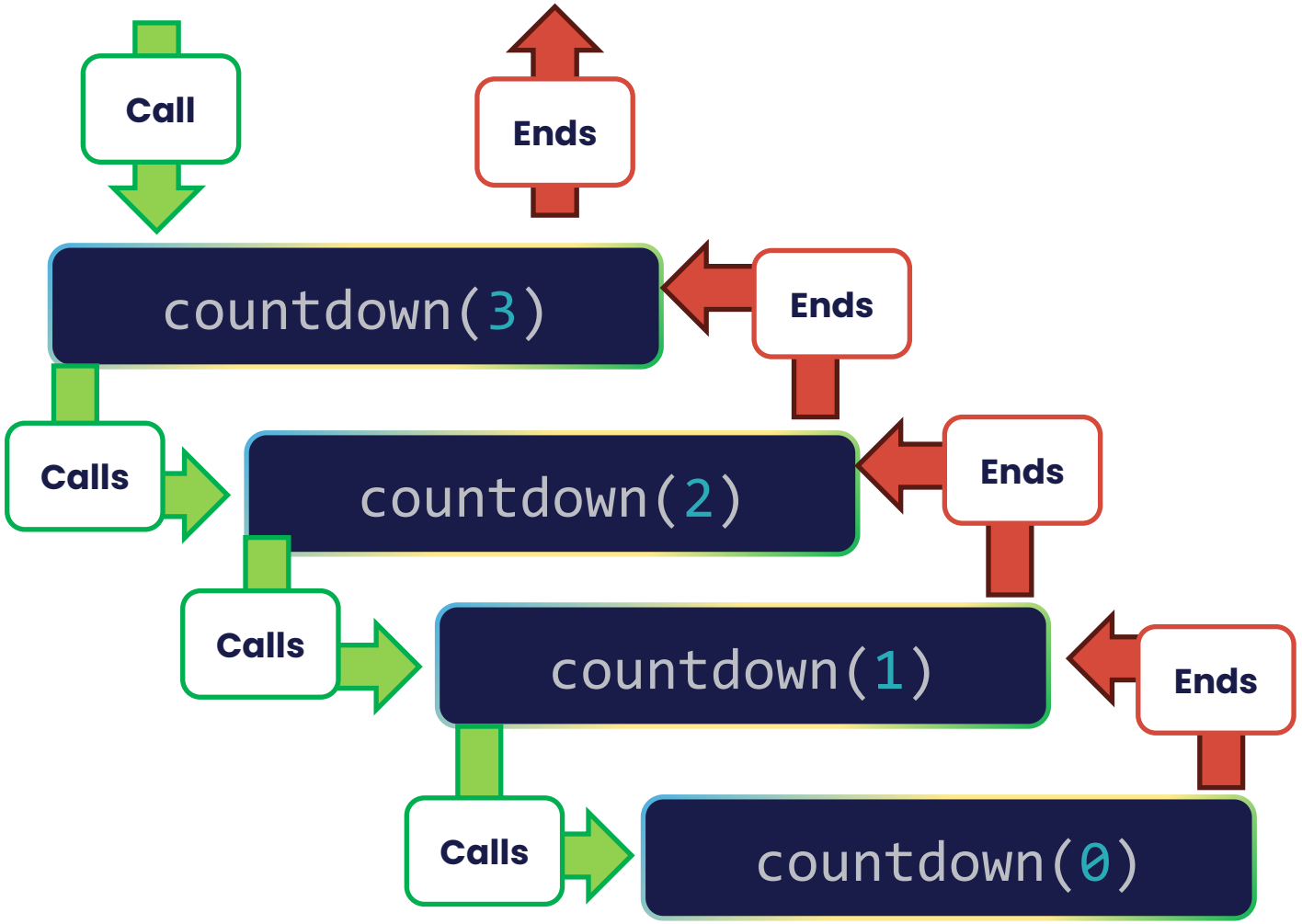
A recursive function can place several unfinished calls on the stack at the same time.

When the base case is reached, those calls finish in reverse order.

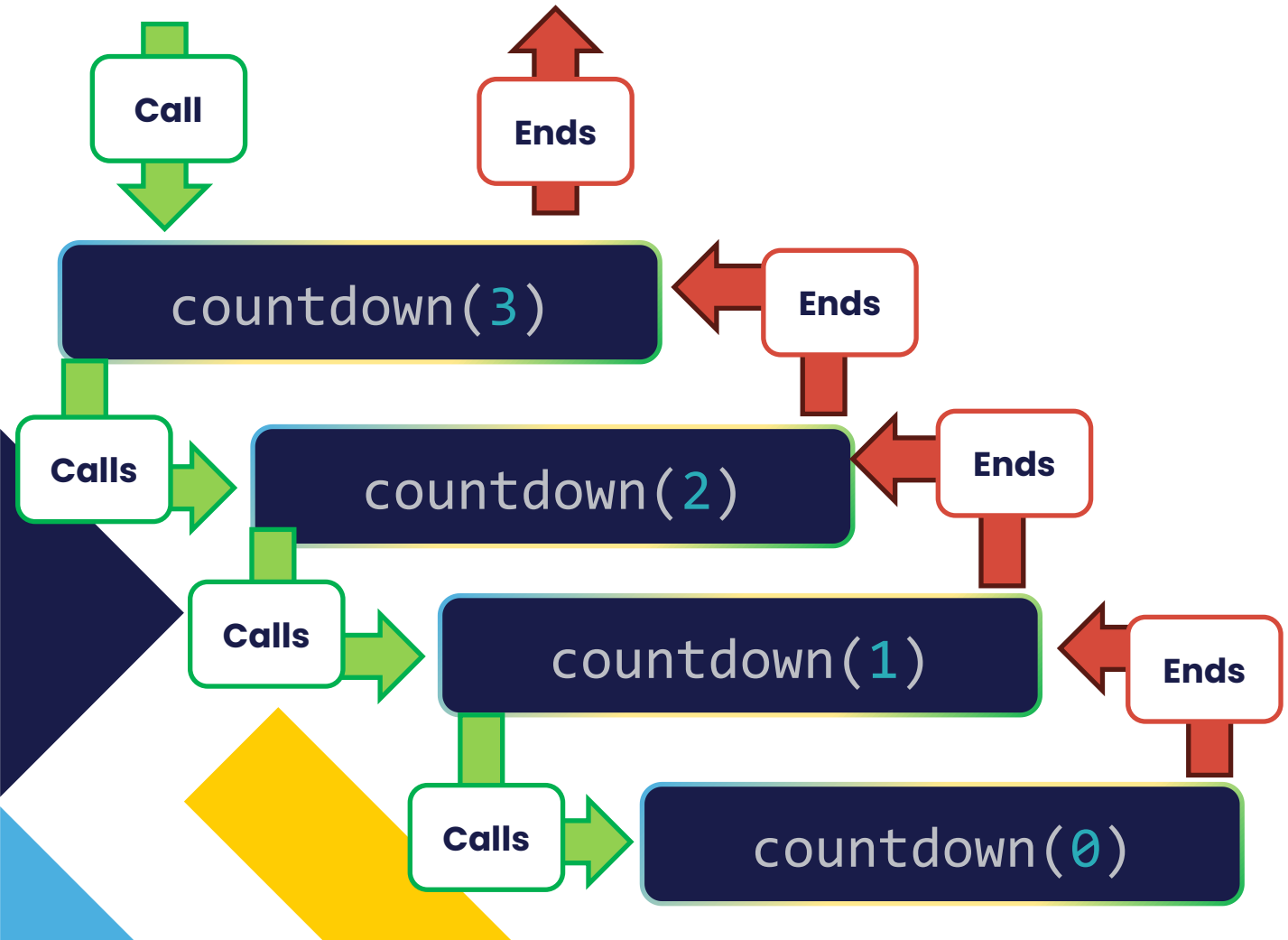
For:

```
countdown(3)
```

the calls build up like this.



# Tail End Recursion - Using the Call Stack




```
def countdown(number):
    # BASE CASE: number == 0
    # Check the base case
    if number == 0:
        return
    # IF NOT THE BASE CASE:
    # Print number
    print(number)
    # RECURSIVE STEP
    # Call function with n - 1
    countdown(number - 1)
    return


countdown(3)
```

# Head End Recursion

**Head end recursion** performs the recursive call before completing the main work of the function.

The function must wait for the recursive call to finish before continuing.

This means the work happens as Python returns back through the call stack.

```
def count_up(number):
    # BASE CASE: number == 0
    if number == 0:
        return

    # RECURSIVE STEP
    count_up(number - 1)

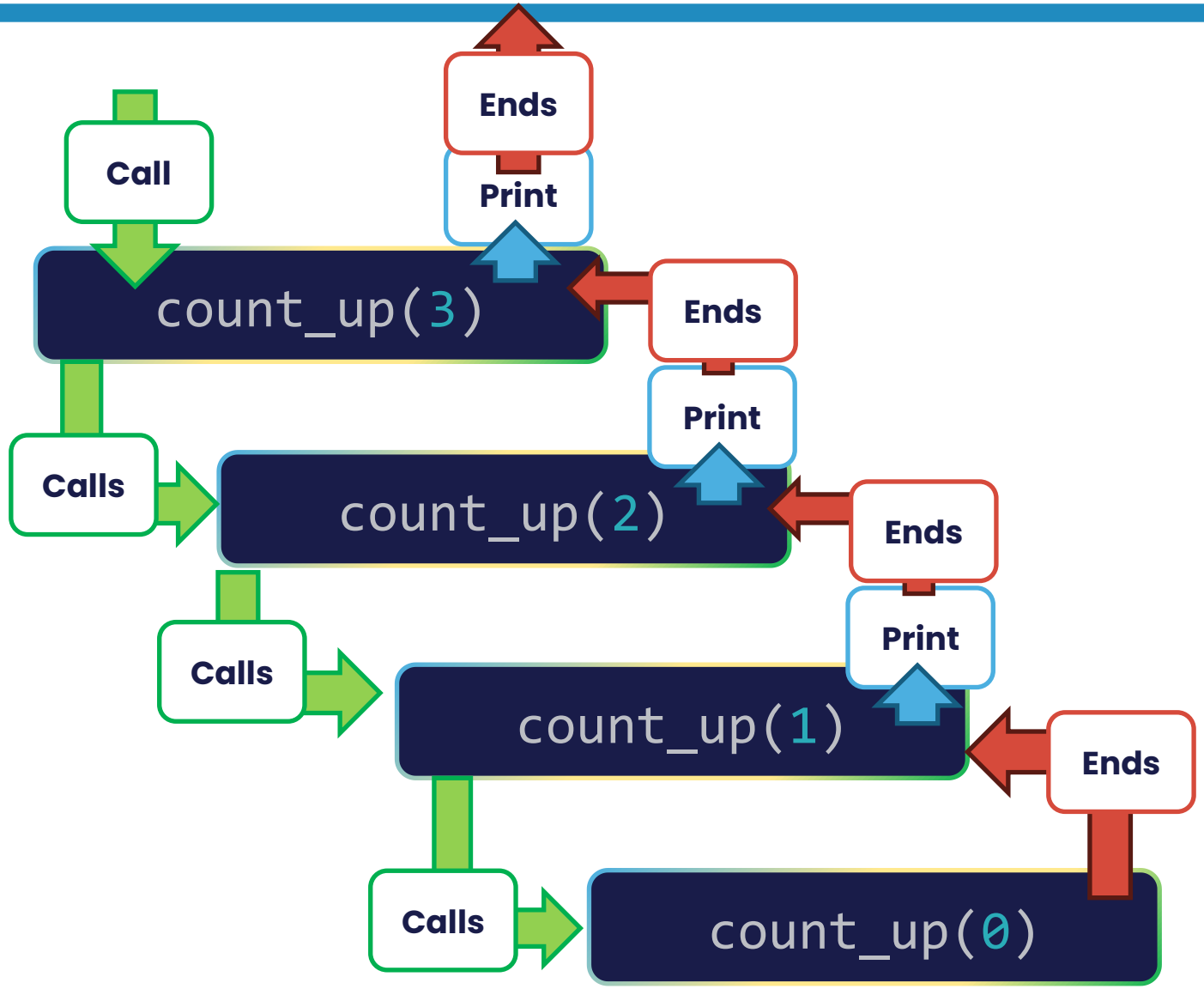
    # PROCESS
    print(number)
    return
```

```
OUTPUT
1
2
3
```

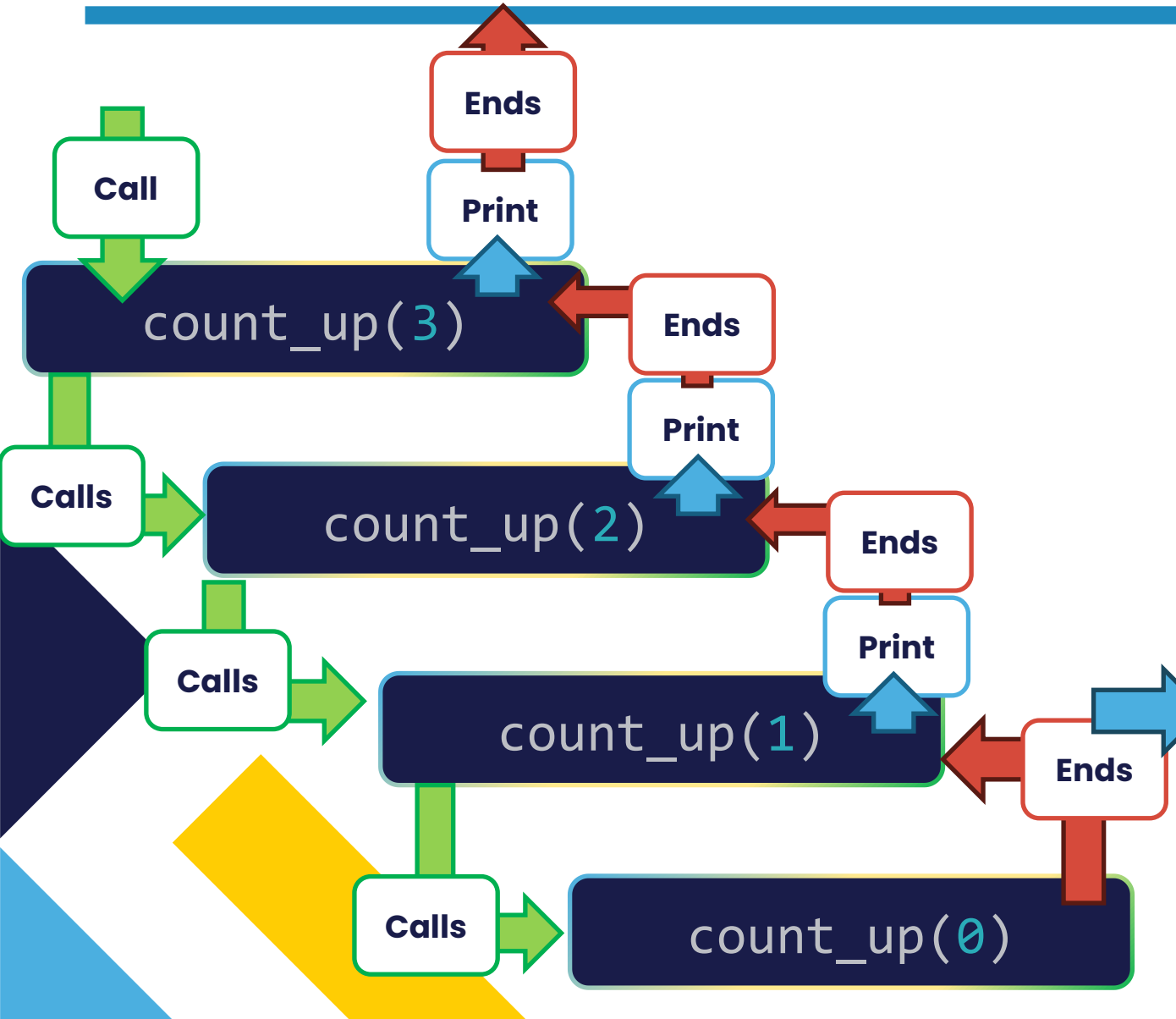
# Head End Recursion - Using the Call Stack

The calls first move downward.

The numbers are printed as the calls finish and return.



# Head End Recursion - Using the Call Stack




```
def count_up(number):
    # BASE CASE: number == 0
    if number == 0:
        return
    # RECURSIVE STEP
    count_up(number - 1)
    # PROCESS
    print(number)
    return
count_up(3)
```

```
OUTPUT
1
2
3
```

# Key Basics to Remember

1. Identify your base case.
2. Implement the base case.
3. Identify the process that needs to be done.
4. Implement the process.
5. Identify how you get closer to the base case.
6. Implement a recursive step that gets closer to the base case.